Landing a rocket in a gust

Jeyaditya Babu, Mohan Raj

In this post, we introduce the concept of Model Predictive Control (MPC), which originated as a mechanism for decision making in the 1960s in the petrochemical industry for complex process control with strict constraints. However, it has found wider applicability, especially in robotic systems, and is now touted as the 'System 2' for complex decision tasks. We begin with the fundamental idea behind MPC, how it is formulated mathematically and then move towards using it in a toy example: vertical landing of a rocket while using as little thrust as possible *à la* SpaceX. Along the way, we will see how prediction, optimization, and constraints come together to form a practical control strategy.

Brief history

In the 1960s and 70s, industrial processes such as those in oil refineries and petrochemical plants had grown too complex with little guarantees to be handled by traditional control methods. A system like a distillation column involved multiple inputs, including reflux rate, reboiler heat, and feed flow, all influencing multiple outputs such as product composition and temperature across different stages. These interactions were tightly coupled, the dynamics were slow, and the effects of any control action could take a long time to appear, all while the system had to operate within strict limits on temperature, pressure, and actuator capability. Engineers tried to manage this using independent PID loops, but these loops acted locally and without foresight, so correcting one variable often disturbed another, leading to oscillations and inefficient operation. At some point it became clear that reacting to the present was not enough, because every action had consequences that unfolded in the future. The question then shifted from what input should be applied now to what sequence of inputs should be applied over time. This change in thinking led to a new approach in which a model of the system is used to predict how it will evolve, an optimization problem is solved over a finite horizon to choose inputs that satisfy

constraints and improve performance, and only the first input is applied before the process is repeated as new information becomes available. This idea, which came to be known as Model Predictive Control, was first implemented in the 1970s in forms such as Dynamic Matrix Control at Shell, where it proved valuable not because it was elegant, but because it worked under real constraints and improved economic performance.

Formulation

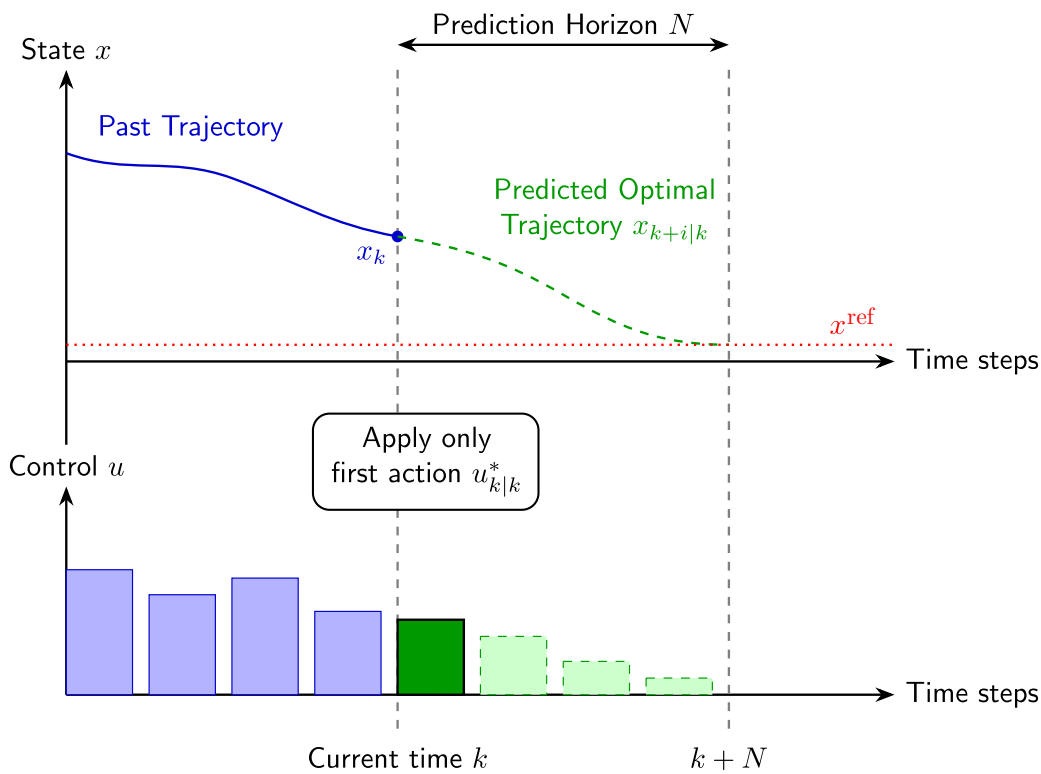

Figure 1: Schematic illustrating the MPC strategy. The top graph shows the past trajectory of the system resulting in the current state x_k, alongside the predicted optimal trajectory approaching the target x^{ref} over the prediction horizon N. The bottom graph displays the corresponding sequence of calculated control inputs. The fundamental idea behind MPC is to compute full future plan but apply only the immediate action $u^*_{k|k}$.

Before we land anything, let us try to understand the mathematical formalism of MPC. Let us consider a system with state x_k (could be the position or momentum

or any variable that characterizes the system of interest) at a discrete time k. The evolution of the state can be written as $x_{k+1} = F(x_k, u_k)$, where u_k is the control input of the system and $F(x_k, u_k)$ is the process that governs the dynamics of the system. Imagine riding a bicycle, the state x_k describes where you are, how fast you are moving, and in what direction you are moving. The control input u_k is the angle to which you rotate the handlebar or the thrust you provide on the pedal. Those actions determine where you will be in the next instant, which is the new state x_{k+1}.

In the framework of Model Predictive Control, our goal is to decide what is the 'best' action to take at a particular instant by considering several future scenarios. That is, at time k, we consider a sequence

$$u_{k|k},\ u_{k+1|k},\ u_{k+2|k}, \ldots$$

to choose the control input u_k. The notation $u_{k+i|k}$ simply means "the control input at a future time $k+i$, computed at time k". Once this future input sequence is fixed, the model tells us exactly what will happen next. Using the model $x_{k+1} = F(x_k, u_k)$, we can predict the next state,

$$x_{k+1|k} = F(x_k, u_{k|k}).$$

Using the same equation again, we can predict the state two steps ahead,

$$x_{k+2|k} = F(x_{k+1|k}, u_{k+1|k}).$$

The state several steps ahead, $x_{k+j|k}$ for $j \gg 1$, depends on the current state, $x_{k|k}$ and on all future control inputs, $u_{k+j|k}$ we have chosen. In other words, once we choose a future control input sequence $u_{k+j|k}$, we have also chosen a future trajectory for the system. Control is not purely reactive, it is about shaping the future trajectory.

The obvious next question is: among all possible future input sequences $u_{k+j|k}$, which one should we choose? To answer this, we introduce a cost function, J. The cost function is a measure of how "good" a predicted future trajectory is with respect to how we want the system to behave. In the rocket landing example, we want the predicted height and velocity to approach zero over time. At the same time, we do not want to use unnecessarily large thrust. A simple way to express this is to penalize two quantities: the rocket's position error and the control effort. Over a horizon of N future steps, we can define J of the form

$$J = \lambda_x \sum_{i=1}^{N} \|x_{k+i|k} - x_{k+i}^{\text{ref}}\|^2 + \lambda_u \sum_{i=0}^{N-1} \|u_{k+i|k}\|^2.$$

In order to find the control input at time-step k, our goal is to minimize J for a fixed horizon N. The optimization problem therefore consists of minimizing J over the future input sequence, subject to the dynamic constraint $x_{k+1} = F(x_k, u_k)$. The first term in the cost function J penalizes the deviation from the desired trajectory while the second penalizes excessive control action. Weighting factors $\lambda_{x,u}$ adjust the relative importance of these two objectives. The predicted outputs depend on the chosen input sequence through the system dynamics. If the dynamics is linear and the cost is quadratic, the resulting optimization problem becomes a well known quadratic program. On the other hand, if the dynamics is nonlinear, the optimization becomes a nonlinear program. Therefore, the optimal sequence of inputs can be found by minimizing J with respect to future inputs. The important aspect of MPC compared to other optimal control frameworks is that after computing this optimal control input $u^*_{k+j|k}$ at time-step k, we do not apply it for all future times from k to $k+N$, rather we apply only the first control action, $u^*_{k|k}$. At the next time-step, we measure the new state x_{k+1}, shift the prediction horizon forward, and solve the optimization problem again. In this way, the control input is continuously re-evaluated based on updated state information. This repeated prediction and optimization is what makes Model Predictive Control adaptive.

In its most general form, a control problem consists of choosing an input sequence u_k that minimizes a performance metric $J(x_k, u_k)$, subject to the system dynamics and physical constraints $x_{k+1} = F(x_k, u_k)$. For a discrete-time system, a finite-horizon optimal control problem can be written as

$$\min_{\{u_0,\ldots,u_{N-1}\}} \sum_{k=0}^{N-1} J(x_k, u_k) + \Phi(x_N)$$

subject to

$$x_{k+1} = F(x_k, u_k).$$

Here, $J(x_k, u_k)$ represents the stage cost accumulated at each step and $\Phi(x_N)$ is the terminal cost. Model Predictive Control operates by repeatedly solving such a finite-horizon optimal control problem in real time. At each time-step, the problem is solved using the current state, x_k as the initial condition, the first control action u_k is applied, we shift the horizon (N steps) by one discrete time step k to $k+1$. In this sense, MPC is not a fundamentally different type of controller but rather a real-time implementation of optimal control. With this background in place, let us now try to land the rocket.

Landing the Rocket

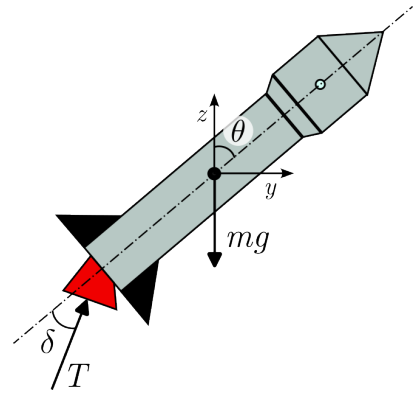

Figure 2: Schematic of the rocket in the yz-plane with its total mass (including fuel) m, orientation θ and control variables, the gimbal angle δ and the thrust T. The mass of the rocket $m(t)$ evolves with time as the fuel gets consumed to exert thrust $T(t)$ while the gimbal angle $\delta(t)$ can be tuned to change the orientation of the rocket.

Consider the task of vertically landing a rocket which, as we shall see, is a concrete instance where we can use the general finite-horizon optimal control framework. Our goal is to plan the motion of a planar rigid rocket with variable mass whose state at time t is given by

$$\mathbf{x}(t) = \begin{bmatrix} y(t) & z(t) & \theta(t) & \dot{y}(t) & \dot{z}(t) & \dot{\theta}(t) & m(t) \end{bmatrix}^\mathsf{T},$$

where $y(t)$ and $z(t)$ denote horizontal and vertical positions, $\theta(t)$ is the pitch angle, $\dot{y}(t)$, $\dot{z}(t)$, and $\dot{\theta}(t)$ are the corresponding velocities, and $m(t)$ is the current mass of the rocket. The control input is

$$\mathbf{u}(t) = \begin{bmatrix} T_{\text{eff}}(t) & \delta(t) \end{bmatrix}^\mathsf{T},$$

where $T_{\text{eff}}(t)$ represents the thrust magnitude and $\delta(t)$ the gimbal deflection angle, which are controlled through actuators. The rocket motion can then be described using Newton's laws for a variable mass rigid body. The thrust acts along the body axis of the rocket, while gravity acts downward (see 2 for a schematic of the setup). The translational dynamics is written as

$$\ddot{y} = \frac{T_{\text{eff}}}{m}\sin(\theta + \delta), \qquad \ddot{z} = \frac{T_{\text{eff}}}{m}\cos(\theta + \delta) - g.$$

The thrust exerted by the rocket, T_{eff} must reach zero when the fuel is empty. This effect is accounted for, while also maintaining the differentiability required by interior-point solvers, by approximating the hard engine cutoff using a smooth function. We define an activation function $\Theta(m)$ that smoothly transitions from 1 to 0 as the mass approaches m_{dry},

$$\Theta(m) = \frac{1}{2}\left[1 + \tanh\left(k(m - m_{\text{dry}})\right)\right],$$

where k is a steepness parameter that controls the sharpness of the cutoff. The effective thrust is then defined as the commanded thrust scaled by this activation factor,

$$T_{\text{eff}}(T, m) = T\Theta(m).$$

The mass depletion dynamics can be written as a single continuous equation,

$$\dot{m} = -\frac{T_{\text{eff}}(T, m)}{v_e}.$$

The translational dynamics driven by this effective thrust are given by

$$\ddot{y} = \frac{T_{\text{eff}}(T, m)}{m}\sin(\theta + \delta), \qquad \ddot{z} = \frac{T_{\text{eff}}(T, m)}{m}\cos(\theta + \delta) - g,$$

and the balance of torque about the center of mass yields the rotational dynamics,

$$\ddot{\theta} = \frac{\ell_{\text{eff}}(m)}{I(m)}\, T_{\text{eff}}(T, m)\sin(\delta).$$

These equations define a nonlinear continuous-time model $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{u})$. To use this model to perform the landing task using MPC, we need to discretize the temporal dynamics using a numerical scheme such as the 4-th order Runge Kutta scheme. This produces a discrete mapping $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k, \mathbf{u}_k)$ given the current state $\mathbf{x}_k$ and the control input $\mathbf{u}_k$. With the model in place, we now identify a cost function that measures performance. For soft landing, position, velocity, and orientation should approach zero near the landing pad. Using the discrete definition of the state,

$$\mathbf{x}_k = \begin{bmatrix} y_k & z_k & \theta_k & \dot{y}_k & \dot{z}_k & \dot{\theta}_k & m_k \end{bmatrix}^\mathsf{T},$$

a reasonable choice of cost function $J(\mathbf{x}_k, \mathbf{u}_k)$ is

$$\begin{aligned} J(\mathbf{x}_k, \mathbf{u}_k) = \sum_{k=0}^{N-1} \Big[& (\mathbf{x}_k - \mathbf{x}_{\text{target}})^\mathsf{T} Q (\mathbf{x}_k - \mathbf{x}_{\text{target}}) + W_T T_k \\ & + R_T T_k^2 + R_\delta \delta_k^2 \Big] + (\mathbf{x}_N - \mathbf{x}_{\text{target}})^\mathsf{T} P (\mathbf{x}_N - \mathbf{x}_{\text{target}}). \end{aligned}$$

The quadratic term $(\mathbf{x}_k - \mathbf{x}_{\text{target}})^\mathsf{T} Q (\mathbf{x}_k - \mathbf{x}_{\text{target}})$ penalizes deviation from the desired landing state, ensuring that position, velocity, and orientation approach zero. The terminal cost $(\mathbf{x}_N - \mathbf{x}_{\text{target}})^\mathsf{T} P (\mathbf{x}_N - \mathbf{x}_{\text{target}})$ enforces the accuracy in the final time step. The term $R_T T_k^2$ penalizes large thrust magnitudes, encouraging smoother control inputs, avoiding aggressive actuation. In addition to this, the linear term $W_T T_k$ directly penalizes the total thrust usage. While the quadratic term discourages large instantaneous thrust, the linear term discourages sustained thrust over time, thereby promoting fuel efficient trajectories.

With the dynamics, cost, and constraints defined, we can now state the landing task explicitly as a finite horizon optimal control problem,

$$\min_{\{\mathbf{x}_k, \mathbf{u}_k\}} J(\mathbf{x}_k, \mathbf{u}_k)$$

subject to

$$\begin{gathered}
\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \ldots, N-1, \\
\mathbf{x}_0 = \mathbf{x}_{\text{measured}}, \\
0 \leq T_k \leq T_{\max}, \qquad |\delta_k| \leq \delta_{\max}, \\
z_k \geq 0, \qquad m_{\text{dry}} \leq m_k \leq m_{\text{wet}}.
\end{gathered}$$

Here, the mass is not directly penalized in the tracking term; fuel efficiency is encouraged through the thrust penalty instead. Solving this problem at each time-step, applying only the first control input, and then repeating the procedure with the updated state yields a receding horizon controller capable of guiding the rocket toward a soft landing while reacting to disturbances along the way. The animation above shows the soft-landing of the rocket using this procedure. The `python`-code used to generate this animation is available here. This toy model was inspired by the problem set in Robert Stengel's book on Optimal Control and Estimation.

Computational Implementation

Translating the continuous-time optimal control problem into code requires a self-consistent discretization procedure. We discretize the dynamics using a 4th-order Runge-Kutta (RK4) scheme with a time step of $dt = 0.1$ s and a horizon of $N = 40$ steps (a 4 sec look-ahead). The resulting Nonlinear Program (NLP) is formulated using `CasADi` library in python. At each time-step, the NLP is solved using `IPOPT` and the solver simultaneously optimizes the physical states $\mathbf{x}_k$, controls $\mathbf{u}_k$ and the

Lagrange multipliers that ensure $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k, \mathbf{u}_k)$ and thrust limits. To achieve real-time feasibility, we use *warm starting*, where we take the optimal control sequence computed at the previous step as the initial guess:

$$\mathbf{u}_k^* = \{\mathbf{u}_{k|k}, \mathbf{u}_{k+1|k}, \ldots, \mathbf{u}_{k+N-1|k}\}$$

We shift this sequence forward by one step and duplicate the final input to create our initial guess:

$$\mathbf{u}_{k+1}^{\text{guess}} = \{\mathbf{u}_{k+1|k}, \mathbf{u}_{k+2|k}, \ldots, \mathbf{u}_{k+N-1|k}, \mathbf{u}_{k+N-1|k}\}$$

We also apply the same shift to the state trajectory $\mathbf{x}_k$ and the lagrange multipliers. This guess places the solver close to the next optimal solution, reducing the computation time and allowing the MPC to run dynamically in real time.

Conclusion

Model Predictive Control extends the capabilities of optimal control but with added adaptability. At every time-step, we use a model of the system to predict how the future might unfold, choose the sequence of inputs that best balances performance and effort, apply only the first input, and then repeat the process with new information. The primary advantage compared to a fixed controller obtained via an optimal control procedure is that you allow the environmental perturbations to affect the system dynamically and adapt to it with an updated controller. The rocket landing problem is only a toy example; however, the MPC framework is general enough that it is today applied in chemical industry, autonomous vehicle planning, active suspension systems, motor drives, wind turbines and wind farm clusters, and several other places.